

《软件开发综合实验》

Comprehensive Experiment on Software Development

詹文翰 高级实验师

zhanwenhan@163. com

课程背景

1. 本课程是电子科技大学计算机专业学生的最后一门**实践类必修课程**，旨在让学生在本科学习的最后阶段能对所学知识进行**综合运用**和**融会贯通**。
2. 课程围绕一个数据备份软件项目，让学生以**团队**为单位，完成从需求分析、系统设计、程序编码到集成测试的**整个软件生命周期**。
3. 通过对软件工程的全流程实践，提升学生的系统分析与建模能力、程序设计与实现能力、团队协作及领导能力，使学生具备足够的面向市场和科研环境的**工程能力**和**职业素养**。

前置课程

《程序设计》
《操作系统》
《软件工程》
《软件配置管理》

《数据结构与算法》
《操作系统环境编程》
《软件开发环境》
《UML统一建模语言》



实验内容及要求

1. 对软件开发过程、软件开发方法、软件开发环境有所掌握并进行基本实践。
2. 设计并实现一款**数据备份软件**，以项目组形式推进，每组最多三人。
3. 基于**面向对象**的软件工程方法学进行项目推进，经历从需求分析、系统设计、编码实现、软件测试的整个**软件生命周期**。
4. 实验最终成果包括一款基本可用的**软件**及其对应文档。
5. 软件应包括选定的完整功能，重点考察其**正确性、易用性、健壮性**。
6. 文档应包括：**需求分析说明书、系统设计文档、软件测试报告**，重点考察其**规范性、一致性、可读性**。
7. 采用现代化**软件开发工具**辅助项目开发，包括但不限于：项目管理工具（Teambition），UML建模工具（StarUML），集成开发环境（VSCode、make、gdb），版本控制工具（git），代码检测工具（linter），软件测试工具（gtest、valgrind、gprof、perf）。



实验评分标准

项目完成分 = 需求分析说明书（10分） +
系统设计文档（20分） +
软件测试报告（20分） +
源码质量（10分） +
项目答辩（10分） +
项目演示（20分） +
AI 辅助编程学习证书（10分）

小组得分 = 项目完成分 * 难度系数

组长得分 = $\text{Min}(100, \text{小组得分})$

组员得分 = $\text{Min}(100, \text{小组得分}-5)$



开发环境

操作系统: **Linux**/MacOS/Windows

开发语言: **C/C++/C#/Java/Go**;

项目要求:

- 1、业务逻辑不能使用脚本语言编写
- 2、对第三方库的使用会影响项目难度系数



AI 的使用

AI 提高了代码产生速度，但没有改变开发者责任

■ Enrichlead: “零手写代码” 上线失败

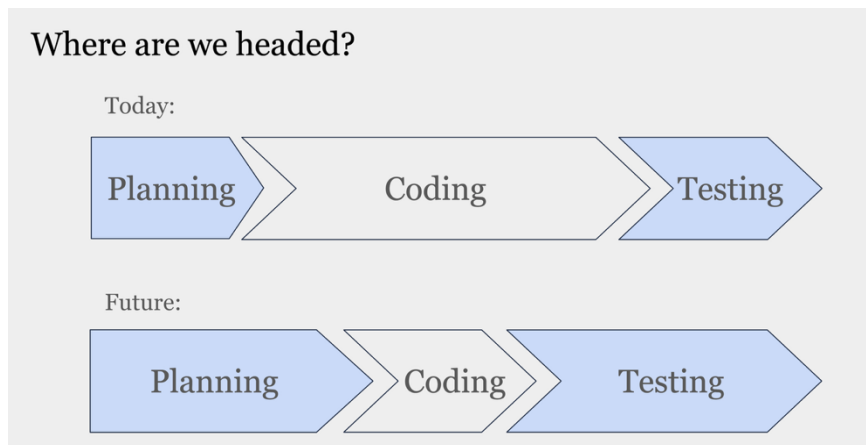
销售线索 SaaS 项目，代码由 Cursor AI 100% 生成
上线数日后发现基础安全缺陷（绕过付费限制 / 修改数据）
未能修复到可接受的安全水平，**项目最终关闭**

■ Slopfix: 重构 “AI 屎山” 的专业团队

收费标准：每周 1 万美元
删除 AI 冗余代码 · 重建工程规范 · 恢复可维护性
典型案例：**10 万行精简至 3.5 万行**，性能明显提升



AI 的使用



人仍需对最终代码负责

读得懂： 理解 C++ 语法、对象关系、接口和设计模式

审得清： 判断 AI 建议是否合理，并对最终代码负责

测得准： 通过编译、单元测试和边界条件验证结果

为此，本课程允许学生使用AI工具辅助完成，但并不鼓励学生一开始就直接使用AI。体验软件开发的整个生命周期以及从 **0** 到 **1** 的开发过程，对打基础阶段非常重要。



提交资料

以小组为单位：

1. 项目报告文档
2. 项目说明PPT
3. AI辅助编程学习证书
4. 源代码+可执行程序
5. 项目演示视频（2分钟以内）

提交方式

- 1-3：提交到实验平台（单独的pdf文档，打包提交）；
4-5：打包提交到助教邮箱。

截止日期

最后一次实验课（第十次课）。

课程安排



自由分组时间

创建班级通知讨论群

关于上课时间

关于实验用机

关于签到

一、需求分析

詹文翰 高级实验师

zhanwenhan@163.com

软件工程方法学

软件危机

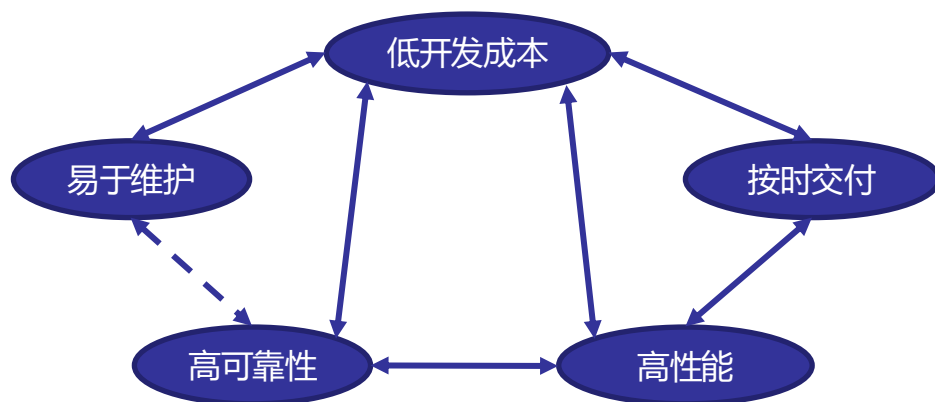
从20世纪60年代末至70年代初，“软件危机”一词在计算机行业广泛流传，指代在软件开发过程和维护中出现的大量问题。这些问题的产生都是由于落后的软件生产方式无法满足迅速增长的计算机软件应用需求所导致。

软件工程

“软件工程”的概念是1968年在德国召开的NATO（北大西洋公约组织）会议上首次提出的，强调应用工程化的原则和方法进行软件的开发和管理。

软件工程方法学

软件工程的目标



软件工程的三要素

软件开发过程：研究软件开发的生命周期，已提出了多种软件开发过程模型，如**瀑布模型**、**原型模型**、**增量模型**等。

软件开发方法：软件开发在分析、设计和编程的过程中所基于的方法学，主流的方法学包括：**结构化方法**和**面向对象方法**。

软件开发工具：为支持不同的软件开发过程和方法学而研制的计算机软件系统和开发环境。

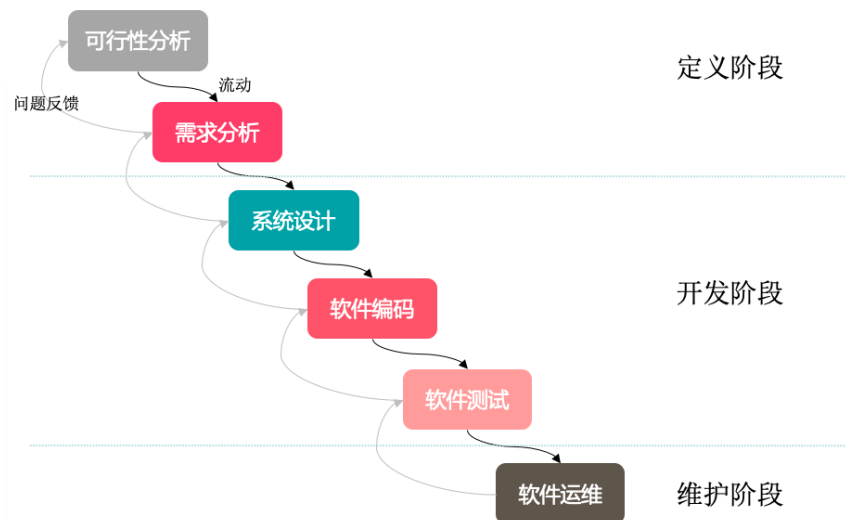
软件过程模型

瀑布模型

瀑布模型（Waterfall Model）是所有软件开发过程模型的基础。其是一个软件项目的**理想**开发架构，开发过程通过一系列的阶段顺序展开，从系统需求分析开始直到产品发布和维护。

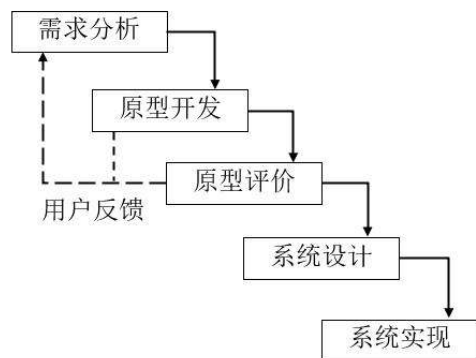
特点：

- 以**里程碑**的方式严格定义各开发阶段的**输入**和**输出**。如果上一阶段的输出达不到要求，则不开展下一阶段的工作。
- 各阶段**定义清晰**，强调文档，便于阶段评审、审计、管理和跟踪。
- 软件生命周期前期造成的Bug的影响比后期的大的多。
- 无法解决软件需求不明确、不准确和完善性问题。

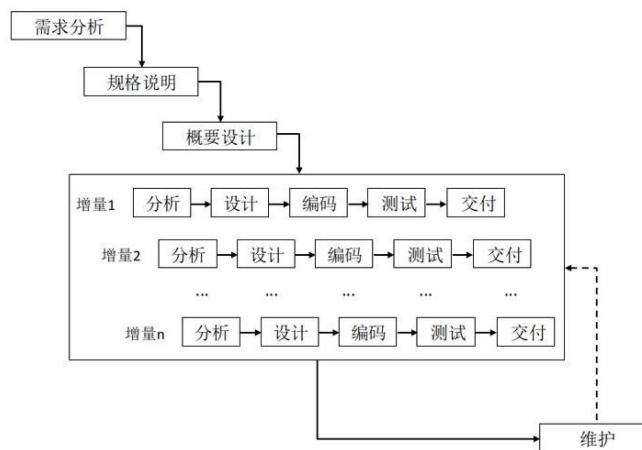


软件过程模型

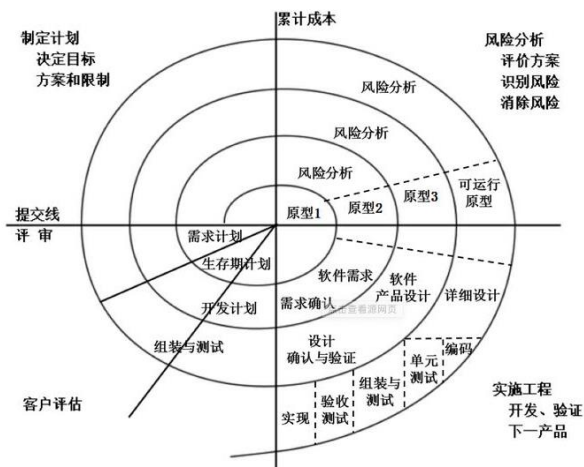
原型模型



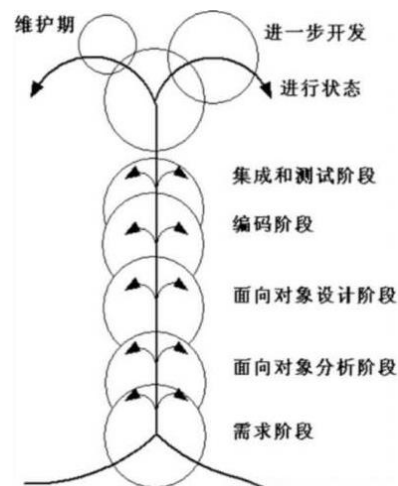
增量模型



螺旋模型



喷泉模型



软件开发方法

结构化方法

最受广大软件工程师所熟知的软件开发方法。该方法是20世纪60年代末在结构化编程的基础上发展而来的，由结构化分析、结构化和结构化编程三部分有机组成。

基本思想：**自顶向下，逐步求精**。通过把一个复杂问题（系统）进行抽象，将大问题（系统）分解为小问题（子系统），再将小问题（子系统）分解为更小的、相对独立的问题（任务）进行实现。

缺点：围绕“**过程**”来构造系统，而非“**功能**”本身；而用户的需求变化主要是针对功能的。因此，需求变化所导致的软件实现代价过大。

软件开发方法

面向对象方法

建立在“对象”概念基础之上，将面向对象的思想应用于软件开发过程中。该方法起源于面向对象的程序设计模式，包括**面向对象分析**、**面向对象设计**和**面向对象编程**三部分。

基本思想：**封装**、**继承**和**抽象**。客观世界由对象组成。将系统中的任何事物都抽象为对象，使开发软件的过程尽可能接近人类的自然思维方式。

由于客观世界的对象之间的关系相对比较**稳定**，因此，面向对象分析和设计的结果也比较稳定，从而在一定程度上提高了软件的**生产率**、**可靠性**、**易重用性**和**易维护性**。

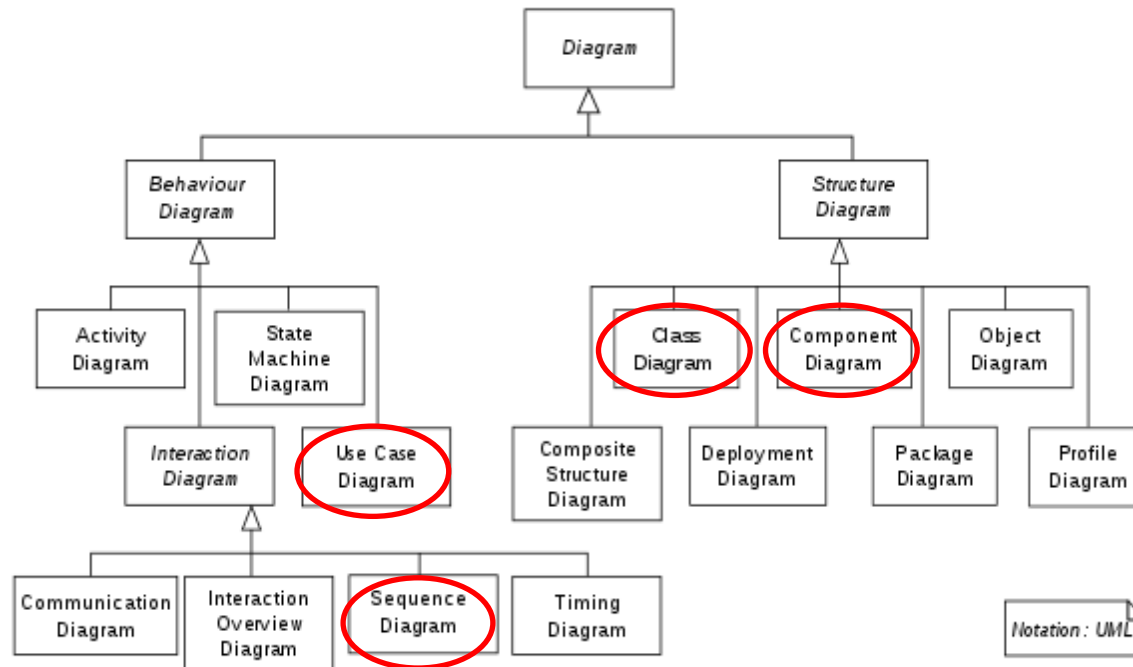
面向对象方法正在逐步成为计算机领域软件开发的主流方法。

统一建模语言UML

面向对象分析与设计的有力工具

UML产生于20世纪90年代中期，是一个**通用的、可视化的**标准建模语言，用于对使用面向对象方法所开发的系统进行说明。

UML是一种**形式化语言**，定义了一系列的图形符号来描述软件系统。这些图形符号有严格的语义和清晰的语法，可以同时**对系统的静态结构和动态结构**进行建模。



面向对象的需求分析

面向对象需求分析的核心是用例模型。**用例模型**是一个包含**参与者**、**用例**及它们之间**关系**的模型，描述了系统应为其用户做些什么以及在哪些约束限定之下。

参与者：指系统以外的，使用系统或与系统产生交互的人或者实体。



用例：一个完整的、可以被参与者感受到的系统功能单元。



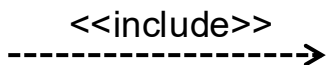
面向对象的需求分析

关系：关联、包含、泛化、扩展四大关系。

关联关系：参与者与用例之间的通信路径。



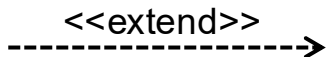
包含关系：在基础用例之上插入附加行为；支持在用例模型中复用功能。



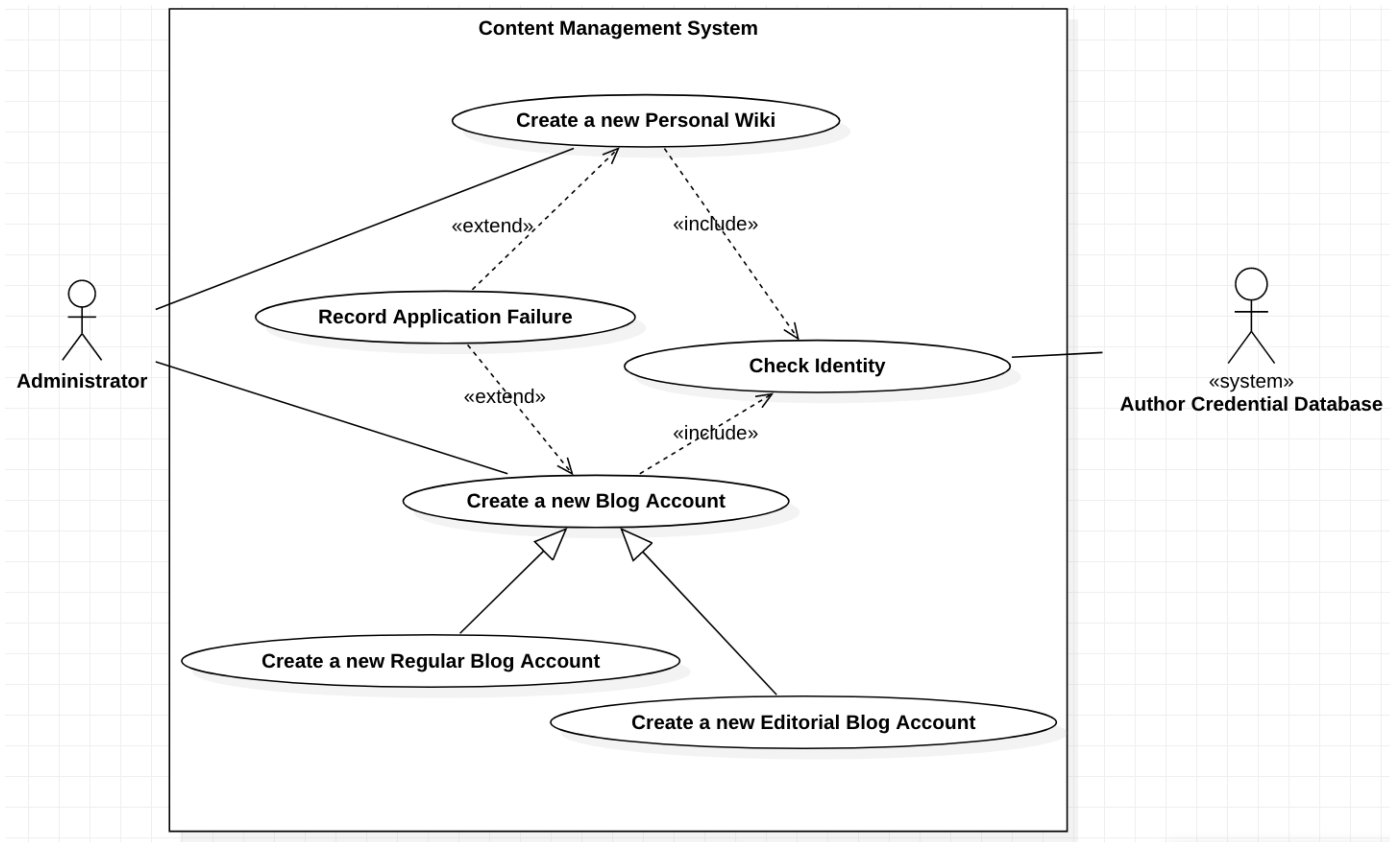
泛化关系：指示子代将接收父代中定义的所有属性、操作和关系，并可以增加新的特性。



扩展关系：对基础用例的功能进行可选的扩充。

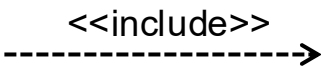
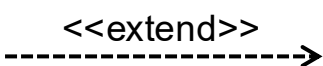


用例图举例



内容发布管理系统（出自《UML学习指南》）

对于关系的进一步说明

包含关系与扩展关系的区别:  

1、**include**通常发生在“多个用例中有可以提取出来的公共部分”，类似于提取公因式。包含方知道有被包含方（箭头指向方）的存在，但被包含方不知道包含方的存在。

2、**extend**表示扩展用例可以扩展被扩展用例（箭头指向方）在某方面的行为，被扩展用例需要留出扩展接口，交由扩展用例来实现。扩展方知道被扩展方的存在，但被扩展方不知道扩展方的存在。

依赖关系: 

正确的用例图是不存在依赖关系的。

依赖关系说明用例之间存在流程上的相互覆盖。针对这种情况，应将用例进一步细化，提取出公共部分，然后使用包含关系（**include**）来表示用例之间的关系。

用例描述

在进行需求分析时，仅用例图并不能提供足够的需求细节，必须假以用例描述，才能让客户以及设计者充分理解真实的需求。

用例描述举例（接上例）：

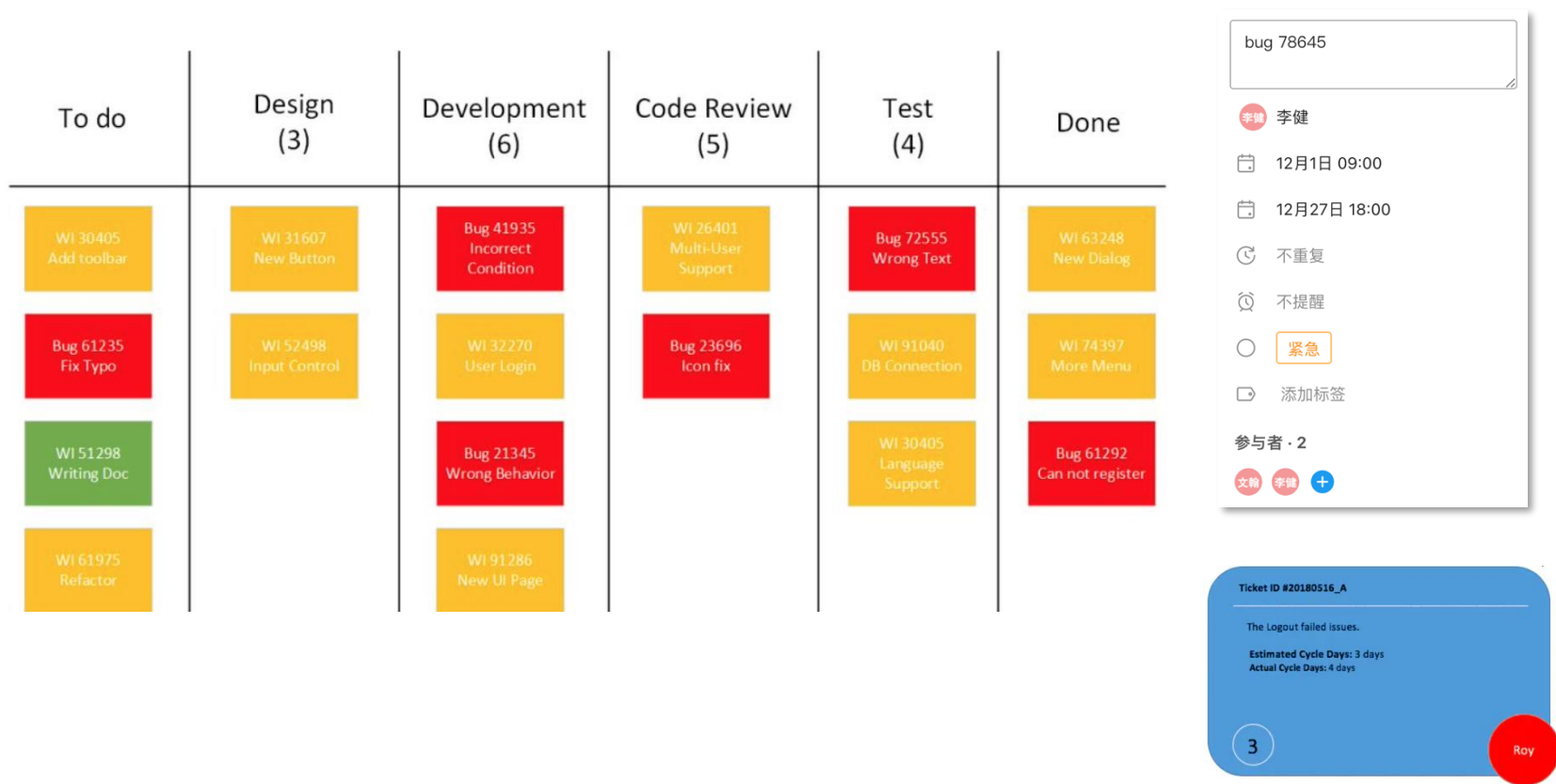
用例标识		xxx	用例名称	Create a new Regular Blog Account
创建人		xxx	创建日期	2020年11月2日
语境目标		一位新的或现有作者通过线下向管理员请求一个新的普通博客账户		
前置条件		作者需有适当的身份证明		
成功的结束状态		成功创建该作者的新普通博客账户		
失败的结束状态		该作者的新普通博客账户申请被拒绝		
参与者		Administrator		
触发器		管理员要求CMS建立一个新的普通博客账户		
基础用例		Create a new Blog Account		
包含用例		Check Identity		
扩展用例		Record Application Failure		
事件流	基本流程	1、管理员要求系统建立一个新的博客账户		
		2、管理员选择普通博客账户类型		
		3、管理员输入作者的详细数据		
		4、检查该作者的详细数据		
		include::Check Identity		
	扩展流程	5、成功创建新的博客账户		
		4.1、不允许该作者建立新的博客		
		4.2、博客账户申请被拒绝		
		4.3、此申请失败情况被记录在该作者的历史数据里		
extend::Record Application Failure				
非功能需求		安全性		
补充规格说明书		无		

项目管理

- 1、看板
- 2、燃尽图
- 3、甘特图
- 4、Scrum
- 5、极限编程

看板

用不同颜色的小纸片代表不同的任务(WI, Work Item), 小纸片上注明任务的各种属性。用一块大的白板(或黑板),在上面画出不同列, 每一列代表任务的一个阶段, 把小纸片放在相应的列下。



看板

最佳实践

- 1、将看板摆在项目组中最显眼的位置。
- 2、为团队每日站会提供讨论依据。
- 3、可以方便的和其它敏捷开发方法做集成。

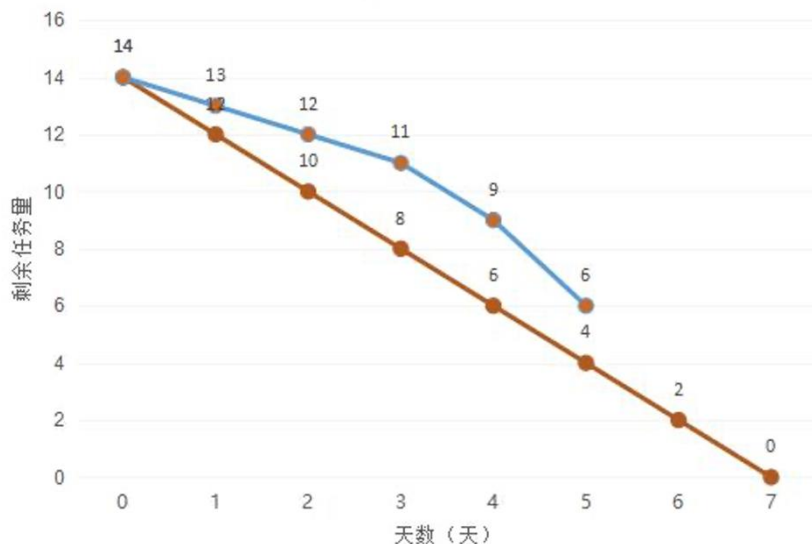


制定和追踪项目计划

燃尽图 (burn down chart)：是用于表示剩余工作量的工作图表，横轴表示时间，纵轴表示剩余工作量，可以直观的预测工作的完成进度，常用于敏捷开发当中。

若实际工作线高于理想工作线时，则意味着剩余的工作量比原先预测的多，并且项目落后于计划。

若实际工作线低于理想工作线，则意味着剩余的工作量比原先预测的少，并且项目提前完成。



制定和追踪项目计划

甘特图（Gantt chart）：通过条状图来显示项目任务和其他相关要素的内在关系随时间进展的情况，以提出者Henry Laurence Gantt的名字命名。

任务：是指软件开发中的一个具体工作。

资源：是指完成任务所需的人员、设备和原材料等。

工期：是完成某项任务所需活动工作时间的总长度，通常是从任务开始日期到完成日期的工作时间量。

里程碑：是一个工期为零，用于标识关键项目节点的重要日程事项。

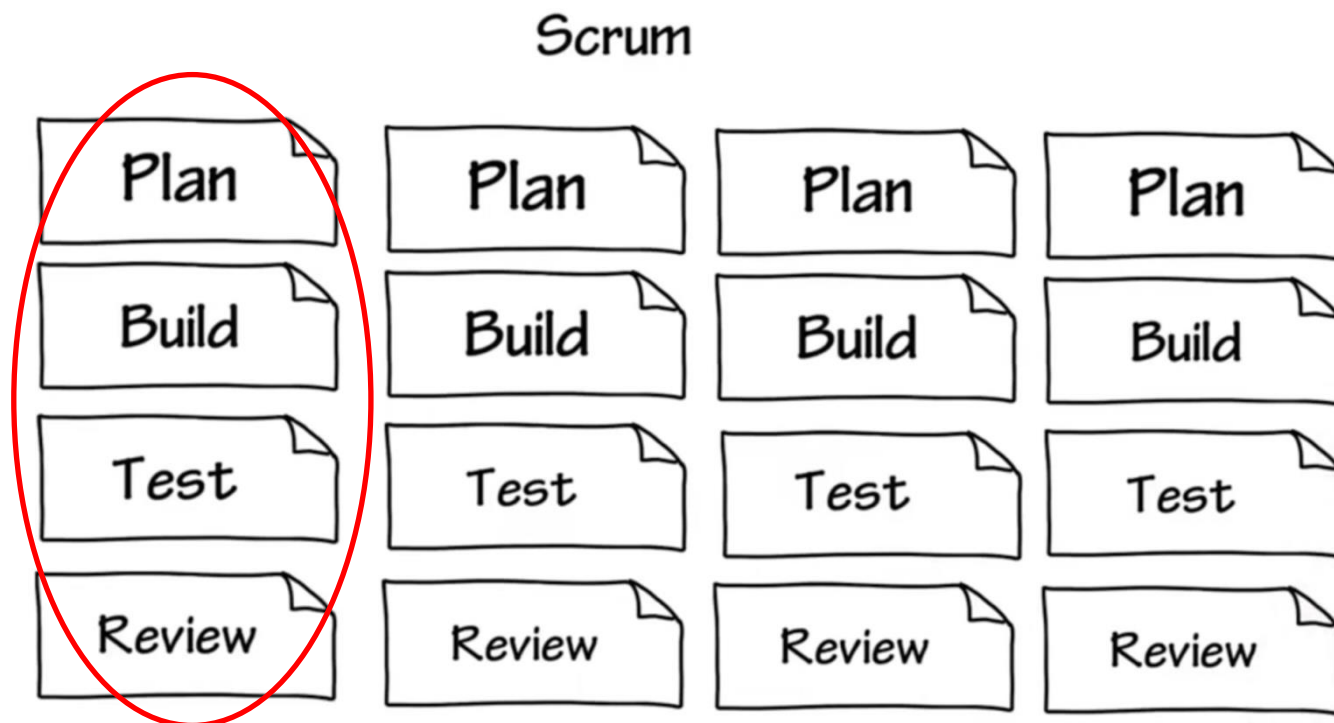


Scrum

最常用的敏捷开发方法之一。

核心思想：小步快跑，逐渐迭代。

Sprint: Scrum把软件开发的过程从时间上分为不同的Sprint（冲刺），每个Sprint的时间可以是1到4周，通常是用2周，一个Sprint包含一个Sprint周期中需要完成的用户功能（user story）。



Scrum核心概念

三种角色：

Product Owner

Team

Scrum Master

三种工具：

User Story

Backlog

Burndown Chart

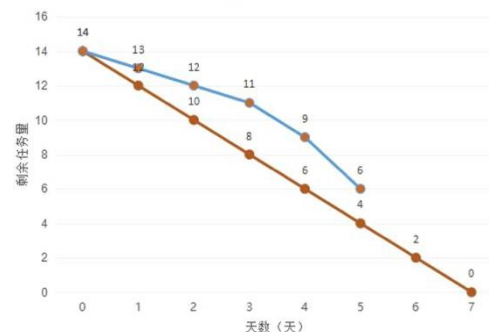
四种会议：

Sprint计划会

每日站会

Sprint评审会

Sprint回顾会



大牛对Scrum的介绍：<https://www.bilibili.com/video/BV1rW411K7vD>

极限编程

另一个常用的敏捷开发方法之一。

核心思想：将目前公认的对编程有益的理念进一步强调并发挥到极致。

举例：

软件测试是重要的



先写测试用例再写代码（TDD）

代码审查是重要的



结对编程

用户反馈是重要的



要求客户直接参与到代码生产中

极限编程

极限编程的12条具体实践：

精细反馈：结对编程、计划博弈、测试驱动开发、现场客户

持续过程：持续集成、代码重构、小型发布

共享认知：编码标准、系统命名隐喻、代码集体共有、简单设计

程序员福利：可持续步调

实验环节

完成以下实验内容：

- 完成实验指导书中指定练习
 - 无需提交实验报告
- 完成本课程项目的需求分析
 - 绘制本项目用例图并撰写用例描述
 - **UML**建模工具任选（**starUML**仅供参考）
 - 撰写用例图中每个用例的用例描述
 - 设计本项目程序界面
 - 界面设计工具任选
 - 对项目进度和分工进行规划
 - 项目管理工具任选（**Teambition**仅供参考）
 - 绘制项目甘特图